# C++ for Quant Interviews: The Complete Guide

From RAII to Low-Latency Systems
Correctness, Memory, and Performance Made Practical

Amit Kumar Jha

January 24, 2026

## Contents

**Practitioner Insight: Vector growth diagram**

Most implementations grow capacity geometrically (often 2x). Reallocation copies/moves all elements, which can dominate latency if done in hot loops. If you know the approximate size, reserve.

### 4.4 Portfolio with metadata: "map to a struct"

Listing 13: Portfolio where each position stores multiple risk fields

```cpp
#include <unordered_map>
#include <string>

struct Position {
  long long qty{};
  double delta{};
  double vega{};
};

using RiskBook = std::unordered_map<std::string, Position>;
```

**The 3-Second Answer: Container choice**

"For market data arrays and time series I default to `vector` because it's contiguous. For a portfolio keyed by symbol I use `unordered_map` for O(1) lookup. I avoid `list` unless I truly need node splicing."

**Cheat Sheet: Module 4**

- Default to contiguous storage: `vector`, `array`.
- Use `deque` for FIFO/rolling windows; avoid `pop(0)` patterns.
- Hash maps for lookup; ordered maps for sorted keys and range queries.
- In performance work, allocations are often the true bottleneck: `reserve`.

# Module 19: Time-Pressure Techniques and Code Review Red Flags

**Module Overview**

- Time pressure: start correct, then optimize. Communicate assumptions explicitly.
- Code review red flags: hidden ownership, unnecessary shared_ptr, unbounded allocations.
- Interview execution: talk in invariants, not syntax.

**3-Second Interview Answer**

Under pressure I write the simplest correct solution, state complexity, then improve locality/allocations if asked.

**Memory Hook**

"Correct $\rightarrow$ clear $\rightarrow$ fast" (in that order).

## 19.1 The 5-step time-pressure protocol

1. Clarify inputs and edge cases (30s).
2. State approach and complexity (30s).
3. Write skeleton (1 min).
4. Fill core logic.
5. Test verbally with small cases.

## 19.2 Code review red flags (C++ edition)

- Raw `new/delete` without strong reason.
- Missing `const` and unclear ownership.
- Returning references to locals.
- Vector growth without `reserve` in known-size loops.
- Silent narrowing conversions (e.g., `double -> int`).
- Exception-unsafe code that leaks resources.

## 19.3 What impresses reviewers

- Clear types (`int64_t` for quantities), named constants (avoid magic 252 without comment).
- RAII everywhere: files, locks, memory.
- Tests with float tolerances and invariants.
- Build instructions and sanitizer targets.

**Cheat Sheet: Module 19**

- Process beats panic: clarify, skeleton, then logic.
- In C++, ownership clarity is the main senior signal.
- Optimize only after correctness and measurement.

## Appendix I: Container Complexity and Invalidation (Safe Rules)

This table is intentionally conservative. In low-level C++, the fastest way to ship a bug is to assume an iterator remains valid after a container mutation. When in doubt: reacquire iterators after modifications, and use `reserve` to control rehash/reallocation.

| Container | Typical lookup | Invalidation rule of thumb (safe) |
|---|---|---|
| `vector` | index O(1) | Reallocation invalidates *all* pointers/refs/iters. Insert/erase invalidates at-and-after position. |
| `deque` | index O(1) | Treat insert/erase/push/pop as invalidating iterators; for portable code, do not store iterators across mutations. |
| `list` | search O(n) | Insert does not invalidate existing iterators/refs; erase invalidates only erased element. Good for stable node addresses, poor for locality. |
| `map` | O(log n) | Insert does not invalidate existing iterators/refs; erase invalidates only erased element. |
| `unordered_map` | avg O(1) | Rehash invalidates *all* iterators/refs/pointers. Inserts may trigger rehash; call `reserve` when possible. Erase invalidates erased only (unless rehash). |

**Practitioner Insight: How to state this in interviews**

"For vectors, I assume any growth can invalidate all references and iterators. For unordered_map, I remember that rehash invalidates everything, so I reserve upfront if the size is known. If I need stable references, I choose a container that guarantees it, or I store indices/IDs instead of iterators."